

https://colab.research.google.com/drive/1ObOa5VGlj8QvqgO_pk-9ImpzNWIIU2Th?authuser=0#scrollTo=_deD7ASM6hd6

Keras; Convolutional Neural Network (CNN); ten-class classifier for CIFAR-10 dataset:

a. Use `cifar10` function in `keras.datasets` to load CIFAR-10 dataset. Split it into the training and testing sets. Define a validation set by randomly selecting 20% of the training images along with their corresponding labels. This will be the “validation_data”.

```
from keras.datasets import cifar10
from sklearn.model_selection import train_test_split

# Load dataset
(x_train_full, y_train_full), (x_test, y_test) = cifar10.load_data()

# Split training set into training and validation (80/20)
x_train, x_val, y_train, y_val = train_test_split(x_train_full, y_train, test
```

b. Scale the pixel values of the images in all the sets to a value between 0 and 1. Perform this process by dividing the image values with 255. Note: No need to flatten the images.

```
x_train = x_train.astype('float32') / 255.0
x_val = x_val.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0
```

c. Convert the label vectors for all the sets to binary class matrices using `to_categorical()` Keras function.

```
from keras.utils import to_categorical

y_train = to_categorical(y_train, num_classes=10)
y_val = to_categorical(y_val, num_classes=10)
y_test = to_categorical(y_test, num_classes=10)

print(x_train.shape)
print(y_train.shape)
```

```
⇒ (40000, 32, 32, 3)
   (40000, 10)
```

d. Using Keras library, build a CNN with the following design: 2 convolutional blocks, 1 flattening layer, 1 FC layer with 512 nodes, and 1 output layer. Each convolutional block consists of two back-to-back Conv layers followed by max pooling. The filter size is 3x3x image_depth. The number of filters is 32 in the first convolutional block and 64 in the second block. Use the following network architecture as a reference:

```
from keras.models import Sequential
from keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout, Activation

# Model architecture
model = Sequential()

# Block 1
model.add(Conv2D(32, (3, 3), padding='same', input_shape=(32, 32, 3)))
model.add(Activation('relu'))
model.add(Conv2D(32, (3, 3), padding='same'))
model.add(Activation('relu'))
model.add(MaxPooling2D(pool_size=(2, 2)))

# Block 2
model.add(Conv2D(64, (3, 3), padding='same'))
model.add(Activation('relu'))
model.add(Conv2D(64, (3, 3), padding='same'))
model.add(Activation('relu'))
model.add(MaxPooling2D(pool_size=(2, 2)))

# Fully connected
model.add(Flatten())
model.add(Dense(512))
model.add(Activation('relu'))
model.add(Dropout(0.5))
model.add(Dense(10, activation='softmax'))

# Compile
from keras.optimizers import Adam
model.compile(optimizer=Adam(learning_rate=0.001),
              loss='categorical_crossentropy',
              metrics=['accuracy'])

# Show model summary
model.summary()
```

⇒ Model: "sequential_2"

Layer (type)	Output Shape	Param #
conv2d_4 (Conv2D)	(None, 32, 32, 32)	896
activation_5 (Activation)	(None, 32, 32, 32)	0
conv2d_5 (Conv2D)	(None, 32, 32, 32)	9,248
activation_6 (Activation)	(None, 32, 32, 32)	0
max_pooling2d_2 (MaxPooling2D)	(None, 16, 16, 32)	0
conv2d_6 (Conv2D)	(None, 16, 16, 64)	18,496
activation_7 (Activation)	(None, 16, 16, 64)	0
conv2d_7 (Conv2D)	(None, 16, 16, 64)	36,928
activation_8 (Activation)	(None, 16, 16, 64)	0
max_pooling2d_3 (MaxPooling2D)	(None, 8, 8, 64)	0
flatten_1 (Flatten)	(None, 4096)	0
dense_2 (Dense)	(None, 512)	2,097,664
activation_9 (Activation)	(None, 512)	0
dropout_1 (Dropout)	(None, 512)	0
dense_3 (Dense)	(None, 10)	5,130

Total params: 2,168,362 (8.27 MB)

Trainable params: 2,168,362 (8.27 MB)

Non-trainable params: 0 (0.00 B)

e. Compile, train, and then evaluate:

Compile the network. Make sure to select a correct loss function for this classification problem. Use Adam optimizer (Adam, learning rate of 0.001).

Use ModelCheckpoint to save the best model based on the lowest validation loss.

Train the network for 50 epochs with a batch size of 32. Remember to assign the validation set to validation_data in the fit function.

Plot the training and validation loss for all the epochs in one plot.

Use the evaluate() Keras function to find the training and validation loss and the accuracy. Report the results.

```
from keras.optimizers import Adam
```

```

from keras.optimizers import Adam
from keras.callbacks import ModelCheckpoint
import matplotlib.pyplot as plt

# Compile
model.compile(loss='categorical_crossentropy',
              optimizer=Adam(learning_rate=0.001),
              metrics=['accuracy'])

# Checkpoint
checkpoint = ModelCheckpoint('best_model.h5', save_best_only=True, monitor='val_loss')

# Train
history = model.fit(x_train, y_train, epochs=50, batch_size=32,
                  validation_data=(x_val, y_val), callbacks=[checkpoint])

# Plot Loss
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.show()

# Evaluate
train_loss, train_accuracy = model.evaluate(x_train, y_train)
val_loss, val_accuracy = model.evaluate(x_val, y_val)
print(f"Training Loss: {train_loss}, Training Accuracy: {train_accuracy}")
print(f"Validation Loss: {val_loss}, Validation Accuracy: {val_accuracy}")

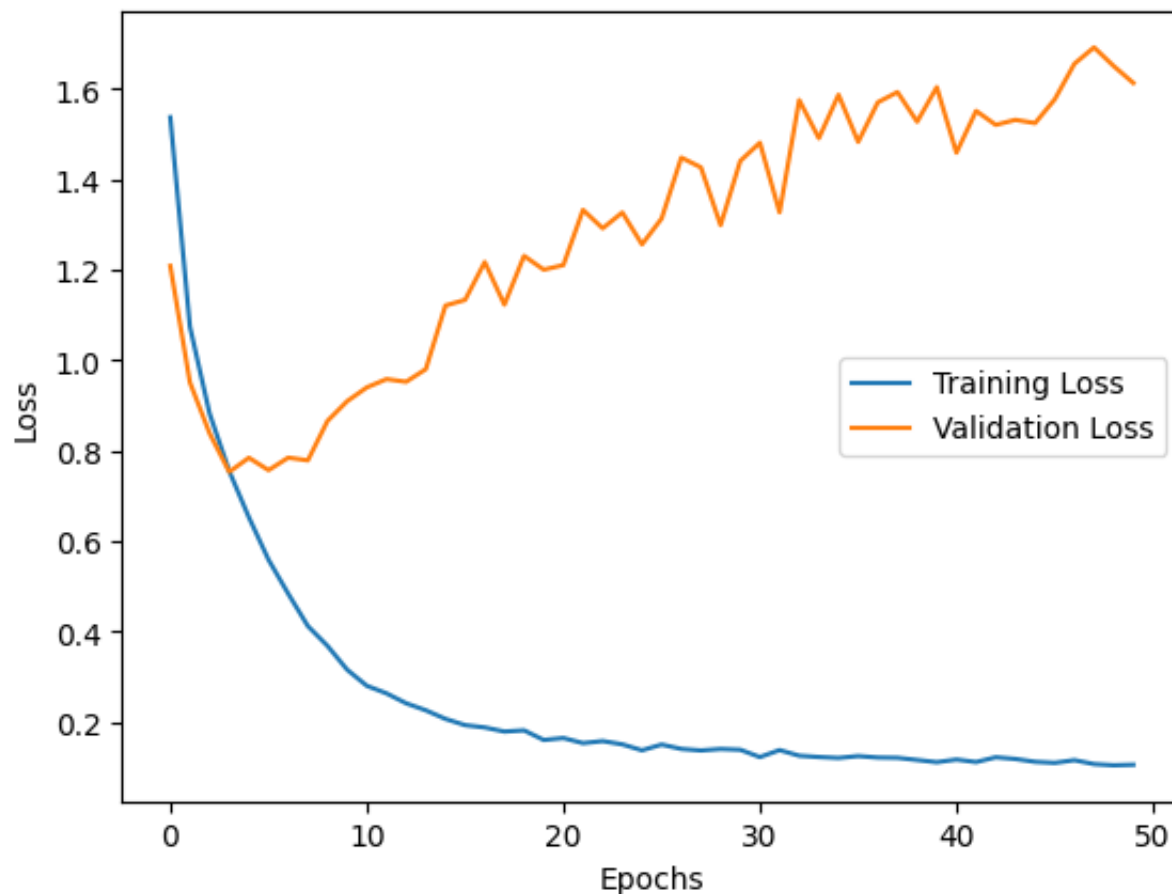
```

```

➡ Epoch 1/50
1249/1250 — 0s 5ms/step - accuracy: 0.3456 - loss: 1.7871
Epoch 1: val_loss improved from inf to 1.20883, saving model to best_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `k
1250/1250 — 11s 6ms/step - accuracy: 0.3458 - loss: 1.7867 -
Epoch 2/50
1245/1250 — 0s 4ms/step - accuracy: 0.6010 - loss: 1.1286
Epoch 2: val_loss improved from 1.20883 to 0.95111, saving model to best_model.
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `k
1250/1250 — 8s 5ms/step - accuracy: 0.6010 - loss: 1.1283 -
Epoch 3/50
1248/1250 — 0s 4ms/step - accuracy: 0.6900 - loss: 0.8890
Epoch 3: val_loss improved from 0.95111 to 0.83856, saving model to best_model.
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `k
1250/1250 — 6s 5ms/step - accuracy: 0.6900 - loss: 0.8890 -
Epoch 4/50
1250/1250 — 0s 5ms/step - accuracy: 0.7370 - loss: 0.7514
Epoch 4: val_loss improved from 0.83856 to 0.75321, saving model to best_model.
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `k
1250/1250 — 7s 6ms/step - accuracy: 0.7370 - loss: 0.7514 -
Epoch 5/50
1250/1250 — 0s 4ms/step - accuracy: 0.7700 - loss: 0.6410

```

Epoch 45/50
1241/1250 — 0s 5ms/step - accuracy: 0.9668 - loss: 0.1076
Epoch 45: val_loss did not improve from 0.75321
1250/1250 — 6s 5ms/step - accuracy: 0.9668 - loss: 0.1076 -
Epoch 46/50
1242/1250 — 0s 4ms/step - accuracy: 0.9682 - loss: 0.0955
Epoch 46: val_loss did not improve from 0.75321
1250/1250 — 6s 5ms/step - accuracy: 0.9682 - loss: 0.0956 -
Epoch 47/50
1246/1250 — 0s 4ms/step - accuracy: 0.9654 - loss: 0.1088
Epoch 47: val_loss did not improve from 0.75321
1250/1250 — 10s 5ms/step - accuracy: 0.9653 - loss: 0.1088 -
Epoch 48/50
1240/1250 — 0s 4ms/step - accuracy: 0.9699 - loss: 0.0924
Epoch 48: val_loss did not improve from 0.75321
1250/1250 — 10s 5ms/step - accuracy: 0.9699 - loss: 0.0925 -
Epoch 49/50
1247/1250 — 0s 5ms/step - accuracy: 0.9680 - loss: 0.1022
Epoch 49: val_loss did not improve from 0.75321
1250/1250 — 7s 6ms/step - accuracy: 0.9680 - loss: 0.1022 -
Epoch 50/50
1250/1250 — 0s 4ms/step - accuracy: 0.9690 - loss: 0.1030
Epoch 50: val_loss did not improve from 0.75321
1250/1250 — 10s 5ms/step - accuracy: 0.9690 - loss: 0.1030 -



1250/1250 — 3s 3ms/step - accuracy: 0.9976 - loss: 0.0095
313/313 — 1s 3ms/step - accuracy: 0.7577 - loss: 1.5589
Training Loss: 0.010064047761261463, Training Accuracy: 0.9974250197410583
Validation Loss: 1.6135579347610474, Validation Accuracy: 0.7524999976158142

f. Now define another model with the same architecture in (d) and then:

Compile the network. Make sure to select a correct loss function for this classification problem. Use Adam optimizer (Adam, learning rate of 0.001).

Use ModelCheckpoint to save the best model based on the lowest validation loss.

Create an image data generator in Keras for real-time data augmentation. The augmentation operations are rotation (10 degrees range), width and height shift (0.1 range), and horizontal flip.

Train the network for 50 epochs with a batch size of 32. Remember to use the image data generator. Assign the validation set to validation_data in the fit function.

Plot the training and validation loss for all the epochs in one plot. Use the evaluate() Keras function to find the training and validation loss and the accuracy. Report the results.

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Activation

def create_model():
    model = Sequential()

    # Block 1
    model.add(Conv2D(32, (3, 3), padding='same', activation='relu', input_shape=(3, 3, 3)))
    model.add(Conv2D(32, (3, 3), padding='same', activation='relu'))
    model.add(MaxPooling2D(pool_size=(2, 2)))

    # Block 2
    model.add(Conv2D(64, (3, 3), padding='same', activation='relu'))
    model.add(Conv2D(64, (3, 3), padding='same', activation='relu'))
    model.add(MaxPooling2D(pool_size=(2, 2)))

    # Flatten and Fully Connected Layers
    model.add(Flatten())
    model.add(Dense(512, activation='relu'))
    model.add(Dropout(0.5))
    model.add(Dense(10, activation='softmax'))

    return model
model_aug = create_model()

# Compile model
model_aug.compile(loss='categorical_crossentropy',
                  optimizer=Adam(learning_rate=0.001),
                  metrics=['accuracy'])

# Create datagen object
```

```

datagen = ImageDataGenerator(
    rotation_range=10,
    width_shift_range=0.1,
    height_shift_range=0.1,
    horizontal_flip=True
)

# Fit to training data (required for some augmentations)
datagen.fit(x_train)
checkpoint_aug = ModelCheckpoint('best_model_aug.h5', save_best_only=True, monitor:

history_aug = model_aug.fit(
    datagen.flow(x_train, y_train, batch_size=32),
    epochs=50,
    validation_data=(x_val, y_val),
    callbacks=[checkpoint_aug]
)
plt.plot(history_aug.history['loss'], label='Train Loss (Aug)')
plt.plot(history_aug.history['val_loss'], label='Val Loss (Aug)')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.title('Training vs Validation Loss with Augmentation')
plt.legend()
plt.show()
train_loss_aug, train_acc_aug = model_aug.evaluate(x_train, y_train)
val_loss_aug, val_acc_aug = model_aug.evaluate(x_val, y_val)

print(f"Train Loss (Aug): {train_loss_aug}, Train Accuracy (Aug): {train_acc_aug}")
print(f"Val Loss (Aug): {val_loss_aug}, Val Accuracy (Aug): {val_acc_aug}")

```

```

➡ Epoch 1/50
1248/1250 ————— 0s 21ms/step - accuracy: 0.3332 - loss: 1.7945
Epoch 1: val_loss improved from inf to 1.29085, saving model to best_model_aug.
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `k
1250/1250 ————— 33s 23ms/step - accuracy: 0.3334 - loss: 1.7940
Epoch 2/50
1250/1250 ————— 0s 20ms/step - accuracy: 0.5449 - loss: 1.2692
Epoch 2: val_loss improved from 1.29085 to 0.98487, saving model to best_model_
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `k
1250/1250 ————— 26s 21ms/step - accuracy: 0.5449 - loss: 1.2692
Epoch 3/50
1250/1250 ————— 0s 20ms/step - accuracy: 0.6065 - loss: 1.1030
Epoch 3: val_loss improved from 0.98487 to 0.89507, saving model to best_model_
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `k
1250/1250 ————— 26s 21ms/step - accuracy: 0.6065 - loss: 1.1030
Epoch 4/50
1249/1250 ————— 0s 20ms/step - accuracy: 0.6420 - loss: 1.0191
Epoch 4: val_loss improved from 0.89507 to 0.83618, saving model to best_model_
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `k
1250/1250 ————— 41s 22ms/step - accuracy: 0.6420 - loss: 1.0190
Epoch 5/50
1249/1250 ————— 0s 21ms/step - accuracy: 0.6730 - loss: 0.9298

```

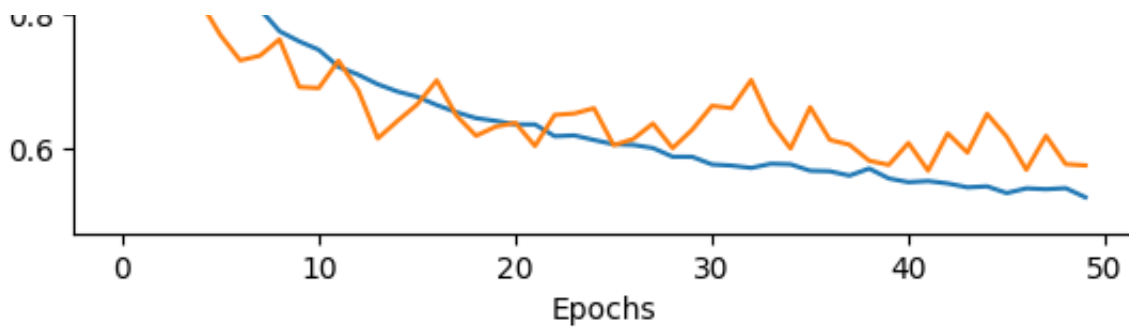
```

1250/1250 ————— 0s 21ms/step - accuracy: 0.8088 - loss: 0.5481
Epoch 42: val_loss improved from 0.57681 to 0.56828, saving model to best_model
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `k
1250/1250 ————— 27s 21ms/step - accuracy: 0.8088 - loss: 0.5481
Epoch 43/50
1250/1250 ————— 0s 20ms/step - accuracy: 0.8104 - loss: 0.5478
Epoch 43: val_loss did not improve from 0.56828
1250/1250 ————— 27s 21ms/step - accuracy: 0.8104 - loss: 0.5478
Epoch 44/50
1249/1250 ————— 0s 21ms/step - accuracy: 0.8158 - loss: 0.5335
Epoch 44: val_loss did not improve from 0.56828
1250/1250 ————— 27s 21ms/step - accuracy: 0.8158 - loss: 0.5335
Epoch 45/50
1248/1250 ————— 0s 21ms/step - accuracy: 0.8094 - loss: 0.5409
Epoch 45: val_loss did not improve from 0.56828
1250/1250 ————— 27s 21ms/step - accuracy: 0.8094 - loss: 0.5409
Epoch 46/50
1250/1250 ————— 0s 20ms/step - accuracy: 0.8172 - loss: 0.5310
Epoch 46: val_loss did not improve from 0.56828
1250/1250 ————— 40s 21ms/step - accuracy: 0.8172 - loss: 0.5310
Epoch 47/50
1249/1250 ————— 0s 21ms/step - accuracy: 0.8139 - loss: 0.5421
Epoch 47: val_loss did not improve from 0.56828
1250/1250 ————— 28s 22ms/step - accuracy: 0.8139 - loss: 0.5421
Epoch 48/50
1249/1250 ————— 0s 22ms/step - accuracy: 0.8119 - loss: 0.5444
Epoch 48: val_loss did not improve from 0.56828
1250/1250 ————— 42s 23ms/step - accuracy: 0.8119 - loss: 0.5444
Epoch 49/50
1248/1250 ————— 0s 21ms/step - accuracy: 0.8152 - loss: 0.5362
Epoch 49: val_loss did not improve from 0.56828
1250/1250 ————— 27s 21ms/step - accuracy: 0.8152 - loss: 0.5362
Epoch 50/50
1248/1250 ————— 0s 20ms/step - accuracy: 0.8157 - loss: 0.5309
Epoch 50: val_loss did not improve from 0.56828
1250/1250 ————— 26s 21ms/step - accuracy: 0.8157 - loss: 0.5309

```

Training vs Validation Loss with Augmentation





1250/1250 ————— 3s 3ms/step - accuracy: 0.8744 - loss: 0.3561
 313/313 ————— 1s 3ms/step - accuracy: 0.8125 - loss: 0.5602
 Train Loss (Aug): 0.35597947239875793, Train Accuracy (Aug): 0.8751749992370605
 Val Loss (Aug): 0.5760852098464966, Val Accuracy (Aug): 0.8112999796867371

g. What do you observe from the validation loss in both step (e) and (f)? Is the model overfitting or underfitting the training data? Explain.

In question e :

Observations:

The training loss still decreases.

The validation loss likely decreases early on, then starts to increase or plateau after a few epochs.

The training accuracy may increase to high values (e.g., >95%), but the validation accuracy lags behind or fluctuates.

This is a characteristic symptom of overfitting. The model performs very well on the training set but fails to generalize to new validation data. That is, it's learning to memorize training set patterns instead of learning more stable features.

In question f :

Observations:

The training loss can decline slower than in (e).

The validation loss is lower and more stable per epoch.

Training accuracy is slightly lower than in (e), but validation accuracy is higher and more consistent.

This illustrates better generalization. The model is no longer overfitting as much since it's now being trained on augmented copies of the data that simulate real-world variations like rotation, shifting, and flipping.

h. Now define another model with the same architecture in (d), except that this time you need to add batch normalization layers to the CNN network.

Add normalization layer after all the convolutional and fully connected layers (not the output layer). Add them before the activation layers and be noted that there is no need for the bias in the convolutional or fully connected layers.

Compile the network. Make sure to select a correct loss function for this classification problem. Use Adam optimizer (Adam, learning rate of 0.01).

Use ModelCheckpoint to save the best model based on the lowest validation loss.

Train the network for 50 epochs with a batch size of 64. Remember to assign the validation set to validation_data in the fit function.

Plot the training and validation loss for all the epochs in one plot.

Use the evaluate() Keras function to find the training and validation loss and

```

from tensorflow.keras.layers import BatchNormalization

def create_bn_model():
    model = Sequential()

    # Block 1
    model.add(Conv2D(32, (3, 3), padding='same', use_bias=False, input_shape=(32, 32, 3)))
    model.add(BatchNormalization())
    model.add(Activation('relu'))
    model.add(Conv2D(32, (3, 3), padding='same', use_bias=False))
    model.add(BatchNormalization())
    model.add(Activation('relu'))
    model.add(MaxPooling2D(pool_size=(2, 2)))

    # Block 2
    model.add(Conv2D(64, (3, 3), padding='same', use_bias=False))
    model.add(BatchNormalization())
    model.add(Activation('relu'))
    model.add(Conv2D(64, (3, 3), padding='same', use_bias=False))
    model.add(BatchNormalization())
    model.add(Activation('relu'))
    model.add(MaxPooling2D(pool_size=(2, 2)))

    # Fully Connected Layer
    model.add(Flatten())
    model.add(Dense(512, use_bias=False))
    model.add(BatchNormalization())
    model.add(Activation('relu'))
    model.add(Dropout(0.5))
    model.add(Dense(10, activation='softmax'))

    return model

model_bn = create_bn_model()
model_bn.compile(optimizer=Adam(learning_rate=0.01),
                 loss='categorical_crossentropy',
                 metrics=['accuracy'])

checkpoint_callback_bn = ModelCheckpoint('model_bn.h5', save_best_only=True, monitor='val_loss')

history_bn = model_bn.fit(
    x_train, y_train,
    validation_data=(x_val, y_val),
    batch_size=64,
    epochs=50,
    callbacks=[checkpoint_callback_bn],
    verbose=2
)

```

```
# Plot loss
plt.plot(history_bn.history['loss'], label='Train Loss (BN)')
plt.plot(history_bn.history['val_loss'], label='Val Loss (BN)')
plt.title('Loss (Batch Normalization)')
plt.legend()
plt.show()
```

```
# Evaluate
train_loss_bn, train_acc_bn = model_bn.evaluate(x_train, y_train, verbose=0)
val_loss_bn, val_acc_bn = model_bn.evaluate(x_val, y_val, verbose=0)
```

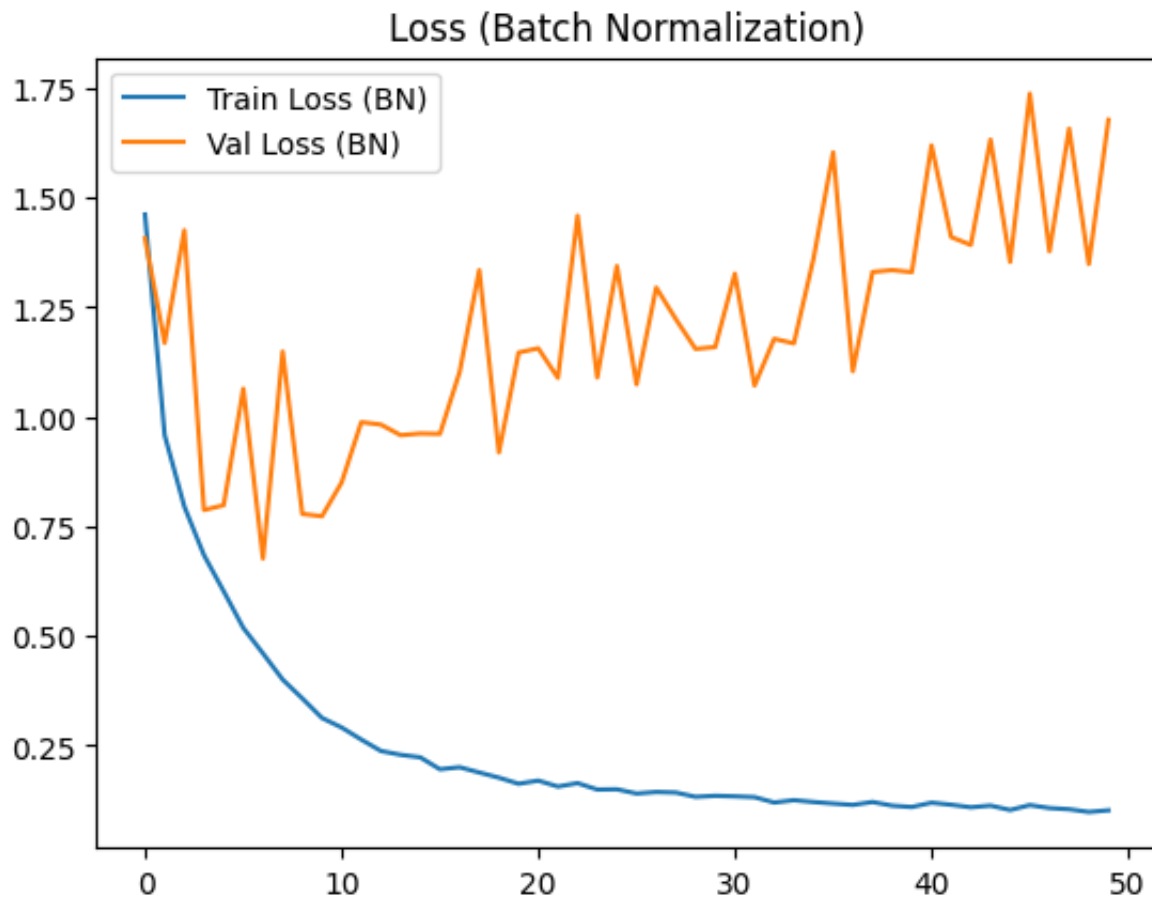
```
print(f"Train Loss (BN): {train_loss_bn:.4f}, Accuracy: {train_acc_bn:.4f}")
print(f"Val Loss (BN): {val_loss_bn:.4f}, Accuracy: {val_acc_bn:.4f}")
```

```
⇒ Epoch 1/50
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `k
625/625 - 13s - 21ms/step - accuracy: 0.4915 - loss: 1.4619 - val_accuracy: 0.5
Epoch 2/50
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `k
625/625 - 5s - 8ms/step - accuracy: 0.6641 - loss: 0.9584 - val_accuracy: 0.596
Epoch 3/50
625/625 - 5s - 8ms/step - accuracy: 0.7226 - loss: 0.7957 - val_accuracy: 0.563
Epoch 4/50
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `k
625/625 - 5s - 8ms/step - accuracy: 0.7617 - loss: 0.6847 - val_accuracy: 0.729
Epoch 5/50
625/625 - 5s - 9ms/step - accuracy: 0.7897 - loss: 0.6025 - val_accuracy: 0.724
Epoch 6/50
625/625 - 5s - 7ms/step - accuracy: 0.8157 - loss: 0.5187 - val_accuracy: 0.678
Epoch 7/50
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `k
625/625 - 5s - 8ms/step - accuracy: 0.8375 - loss: 0.4605 - val_accuracy: 0.776
Epoch 8/50
625/625 - 4s - 7ms/step - accuracy: 0.8601 - loss: 0.4009 - val_accuracy: 0.667
Epoch 9/50
625/625 - 5s - 7ms/step - accuracy: 0.8753 - loss: 0.3579 - val_accuracy: 0.764
Epoch 10/50
625/625 - 5s - 8ms/step - accuracy: 0.8875 - loss: 0.3133 - val_accuracy: 0.766
Epoch 11/50
625/625 - 5s - 7ms/step - accuracy: 0.8973 - loss: 0.2915 - val_accuracy: 0.776
Epoch 12/50
625/625 - 5s - 8ms/step - accuracy: 0.9081 - loss: 0.2640 - val_accuracy: 0.754
Epoch 13/50
625/625 - 4s - 7ms/step - accuracy: 0.9180 - loss: 0.2379 - val_accuracy: 0.776
Epoch 14/50
625/625 - 4s - 7ms/step - accuracy: 0.9212 - loss: 0.2292 - val_accuracy: 0.775
Epoch 15/50
625/625 - 5s - 9ms/step - accuracy: 0.9235 - loss: 0.2234 - val_accuracy: 0.769
Epoch 16/50
625/625 - 5s - 8ms/step - accuracy: 0.9333 - loss: 0.1962 - val_accuracy: 0.767
Epoch 17/50
625/625 - 5s - 8ms/step - accuracy: 0.9316 - loss: 0.2008 - val_accuracy: 0.768
Epoch 18/50
```

```

Epoch 45/50
625/625 - 5s - 8ms/step - accuracy: 0.9693 - loss: 0.1031 - val_accuracy: 0.781
Epoch 46/50
625/625 - 5s - 8ms/step - accuracy: 0.9658 - loss: 0.1144 - val_accuracy: 0.771
Epoch 47/50
625/625 - 5s - 8ms/step - accuracy: 0.9677 - loss: 0.1076 - val_accuracy: 0.795
Epoch 48/50
625/625 - 5s - 7ms/step - accuracy: 0.9684 - loss: 0.1049 - val_accuracy: 0.792
Epoch 49/50
625/625 - 5s - 8ms/step - accuracy: 0.9710 - loss: 0.0987 - val_accuracy: 0.800
Epoch 50/50
625/625 - 5s - 7ms/step - accuracy: 0.9699 - loss: 0.1025 - val_accuracy: 0.794

```



```

Train Loss (BN): 0.0129, Accuracy: 0.9961
Val Loss (BN): 1.6780, Accuracy: 0.7949

```

i. What do you observe from the training loss in both steps (e) and (h)? Explain.

In question e: CNN without Batch Normalization

Observations:

The training loss decreases sharply in the first few epochs.

It has a tendency to reach a very small value (close to zero).

Training accuracy becomes extremely high (usually more than 95%).

On the contrary, validation loss can start increasing or fluctuate, which means overfitting.

This sharp decline in training loss tells us that the model is learning very aggressively and adapting to the training data highly – but possibly not in a generalizable fashion. This sort of behavior typically indicates overfitting.

Question h: CNN with Batch Normalization (BN)

Observations:

Training loss decreases more gradually compared to (e).

It does not always decrease as low as value as in (e), but levels off.

The validation loss is more stable and lower than in step (e).

Training and validation accuracy tends to converge closer, indicating improved generalization.

The model gets smoothed and trains more uniformly using batch normalization. BN normalizes the activations after each of the convolution and fully connected layers (before applying the activation function), regularizing the learning process. It prevents sudden weight update oscillations and prevents the model from converging on sharp minima that are not generalizable.